

System Administrator

Infrastructure · Deployment · Maintenance · Troubleshooting

DOCUMENT VERSION

ISSUE DATE

TARGET AUDIENCE

CLASSIFICATION

SYSTEM

PLATFORM



Table of Contents

- System Overview §1
- Architecture Diagram §2
- Server Requirements §3
- Environment Variables (.env) §4
- Apache Virtual Hosts §5
- Database — Local MySQL & Remote PostgreSQL §6
- Billing Date Policy — The 28th Rule §7
- Deployment & Upgrade Procedures §8
- Logs & Monitoring §9
- Troubleshooting Reference §10
- Security Checklist §11



System Overview

What the Smart Metering Portal does and how it fits in the HWA ecosystem

The **Hargeisa Water Agency Smart Metering Portal** (internally referred to as the *Nobleus Portal*) is the central digital operations platform for managing IoT-based smart water meters deployed across Hargeisa. It ingests live telemetry from the ThingsBoard IoT platform, stores readings locally in a MySQL database hosted on AWS EC2, and synchronises billing data to the HWA Core Billing system running on-premise via a secured REST API.

The system follows a hub-and-spoke integration model: all field devices report to ThingsBoard, which pushes webhook payloads to the portal. The portal normalises, validates, and enriches the data before forwarding relevant billing entries to the Core Billing API at `reading.hargeisawatertech.com`. A FortiGate firewall VIP at **102.218.58.66** provides network-level separation between the AWS environment and the on-premise HWA LAN.

Public-Facing Domains

<code>portal.hargeisawatertech.com</code>	Full portal — meter management, billing, reports, user admin	<code>/var/www/nobleus-portal/web</code>	LET'S E
<code>billing.hargeisawatertech.com</code>	Billing-only view for finance department staff	<code>/var/www/nobleus-portal/web</code>	LET'S E
<code>api.hargeisawatertech.com</code>	REST API endpoint for external consumers (mobile app, integrations)	<code>/var/www/nobleus-portal/web</code>	LET'S E
<code>reading.hargeisawatertech.com</code>	HWA Core Billing API — external service, not served by this server	<i>On-premise HWA server</i>	HWA-M

Technology Stack

Application Framework	Yii 2 (PHP)	PHP 8.3, Yii 2.0.x
Web Server	Apache HTTP Server	2.4 — mod_rewrite, mod_ssl, mod_headers
Local Database	MySQL	8.0 — InnoDB engine, UTF8MB4
Remote Database	PostgreSQL	10+ (on-premise HWA server)
Cloud Host	AWS EC2	Ubuntu 22.04 LTS, t3.medium+
IoT Platform	ThingsBoard	REST webhook integration
SSL Certificates	Let's Encrypt / Certbot	Auto-renewal via cron
Dependency Manager	Composer	2.x
Source Control	Git	2.x — deployment via git pull

Architecture Diagram

End-to-end data flow from IoT field devices to HWA on-premise billing database

The diagram below illustrates the complete data path from smart meters in the field through to the HWA PostgreSQL billing database. All inter-service communication is encrypted. The only inbound path into the AWS EC2 instance is HTTPS (443); outbound calls to the Core Billing API cross the internet to the FortiGate VIP which proxies traffic to the internal application server.



IoT Smart Meters
Pulse / AMR / LoRa
Field devices citywide

FIELD

MQTT / LoRa



ThingsBoard IoT Platform
Device registry
Telemetry aggregation
Rule engine

IOT CORE

HTTPS REST Webhook
TB_WEBHOOK_TOKEN



Nobleus Portal
AWS EC2 · Ubuntu 22.04
PHP 8.3 / Yii 2
Apache 2.4

AWS EC2

TCP 3306
localhost



MySQL 8.0
Local database
nobleus_db
meter_reading_raw ...

LOCAL DB

HTTPS · JWT (CORE_API_TOKEN)
CoreBillingClient





Core Billing API

reading.hargeisawatertech.com
HWA-managed REST service
JWT-authenticated

EXT. AP



FortiGate VIP · 102.218.58.66

NAT → 10.10.50.120



FortiGate Firewall

VIP 102.218.58.66
Policy-based routing
SSL inspection

PERIMETER

TCP 5432
10.10.50.120



HWA PostgreSQL DB

On-premise server
HWA_DATA database
user: openpg

ON-PREM

i

NETWORK BOUNDARY

The FortiGate VIP at 102.218.58.66 is the only internet-facing ingress point for the HWA on-premise network. The Core Billing API application at 10.10.50.120 is never directly reachable from the internet — all traffic passes through FortiGate policy inspection first.



Server Requirements

Minimum and recommended stack for production deployment

The portal is designed to run on a single AWS EC2 instance in a production-grade configuration. The specifications below reflect the minimum requirements for stable operation with up to 500 active smart meters; scale up the instance type for larger deployments.

Operating System	Ubuntu 22.04 LTS (Jammy)	Latest point release	LTS ensures 5-year security support until April 2027
Web Server	Apache HTTP Server 2.4+	2.4.52+	Requires <code>mod_rewrite</code> , <code>mod_ssl</code> , <code>mod_headers</code> , <code>mod_deflate</code>
PHP	PHP 8.1 or PHP 8.3	PHP 8.3 (active)	Extensions: <code>pdo_mysql</code> , <code>curl</code> , <code>mbstring</code> , <code>intl</code> , <code>gd</code> , <code>zip</code> , <code>xml</code>
Local Database	MySQL 8.0+	8.0.x	InnoDB, UTF8MB4, local TCP 3306; bind to <code>127.0.0.1</code> or <code>:::tcp6:::3306</code>
Remote Database	PostgreSQL 10+	10+ (HWA-managed)	Outbound TCP 5432 to <code>102.218.58.66</code> must be allowed
Composer	Composer 2.x	2.x latest	Used for PHP dependency management at deploy time
Git	Git 2.x	2.34+	Used for code deployment; SSH deploy key or HTTPS token
SSL Certificates	Let's Encrypt via Certbot	Latest Certbot	Auto-renewal cron required; see Section 5
EC2 Instance Type	t3.medium minimum	t3.large for production	2 vCPU, 4 GB RAM minimum; 8 GB RAM recommended
Storage	20 GB root EBS volume	50 GB gp3	Monitor disk usage; Apache and Yii logs can grow significantly

PHP Extension Verification

After installation, verify all required PHP extensions are loaded:

```
bash — verify PHP extensions

ubuntu@hwa-portal:~$ php -m | grep -E "pdo_mysql|curl|mbstring|intl|gd|zip|xml|openssl"
curl
gd
intl
mbstring
openssl
pdo_mysql
xml
zip
ubuntu@hwa-portal:~$ php -v
PHP 8.3.x (cli) (built: ...) ( NTS )
Copyright (c) The PHP Group
Zend Engine v4.3.x, Copyright (c) Zend Technologies
    with Zend OPcache v8.3.x, Copyright (c), by Zend Technologies
```



Environment Variables (.env)

Complete reference for all application configuration keys

All environment-specific configuration is managed through a `.env` file in the application root (`/var/www/nobleus-portal/.env`). This file is loaded at bootstrap by the Yii 2 application. It is **never committed to Git** — the repository contains a `.env.example` template only.



SECURITY CRITICAL

Never commit `.env` to any Git repository. The file contains database credentials, API tokens, and JWT secrets. Set file permissions to `640` and owner to `www-data:www-data`. Rotate all tokens at least every 6 months as per the security policy.

Full .env Reference

`/var/www/nobleus-portal/.env`

```
# =====
# HWA Smart Metering Portal — Environment Configuration
# Copy this file from .env.example and fill in production values.
# Owner: www-data:www-data  Permissions: 640
# =====

# — Application —
YII_ENV=prod
YII_DEBUG=false
APP_NAME="HWA Smart Metering Portal"

# — Local MySQL Database —
DB_HOST=127.0.0.1
DB_PORT=3306
DB_NAME=nobleus_db
DB_USER=nobleus_app
DB_PASSWORD=YOUR_STRONG_MYSQL_PASSWORD

# — ThingsBoard IoT Integration —
# Webhook token — must match the token configured in the TB rule engine
TB_WEBHOOK_TOKEN=YOUR_TB_WEBHOOK_SECRET_TOKEN
# ThingsBoard base URL (include trailing slash)
TB_URL=https://iot.hargeisawatertech.com/
# ThingsBoard API credentials (for reading device telemetry)
TB_USERNAME=api-service@hwa.so
TB_PASSWORD=YOUR_TB_API_PASSWORD
# The telemetry key name used by meters to report cubic-metre readings
TB_TELEMETRY_KEY=volume_m3

# — Portal / API URLs —
BILLING_PORTAL_URL=https://billing.hargeisawatertech.com
PORTAL_URL=https://portal.hargeisawatertech.com
API_BASE_URL=https://api.hargeisawatertech.com

# — HWA Core Billing API —
# Primary endpoint: reading ingestion and billing sync
CORE_READING_API_URL=https://reading.hargeisawatertech.com
# Meters meta-data endpoint
CORE_METERS_API_URL=https://reading.hargeisawatertech.com/api/meters
# JWT bearer token for Core API authentication — rotate every 6 months
CORE_API_TOKEN=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...YOUR_JWT_HERE
# Optional: set to 1 ONLY for local development to bypass SSL verification
# MUST be 0 or unset in production
CORE_API_INSECURE_SSL=0
```

Key Descriptions

YII_DEBUG	CRITICAL	Must be <code>false</code> in production. Debug mode exposes stack traces publicly.
TB_WEBHOOK_TOKEN	REQUIRED	Shared secret validating inbound ThingsBoard webhook calls. Used as Bearer token.
TB_TELEMETRY_KEY	REQUIRED	JSON key in ThingsBoard telemetry payload containing the cubic-metre reading value.
CORE_API_TOKEN	REQUIRED	JWT used as <code>Authorization: Bearer</code> header on all calls to the Core Billing API.
CORE_API_INSECURE_SSL	OPTIONAL	Set to <code>1</code> only in dev environments with self-signed certs. Never <code>1</code> in production.



TOKEN ROTATION POLICY

`CORE_API_TOKEN` (JWT) and `TB_WEBHOOK_TOKEN` must be rotated every 6 months or immediately upon any suspected compromise. Update the `.env` file and reload Apache (`sudo systemctl reload apache2`) — no application restart is required for environment variable changes.



Apache Virtual Hosts

Virtual host configuration, SSL setup, and catch-all protection

The portal uses Apache 2.4 with multiple named virtual hosts to serve different sub-domains from the same EC2 instance. All virtual host configuration files reside in `/etc/apache2/sites-available/` and are symlinked via `a2ensite`.

Virtual Host File Inventory

portal.conf	portal.hargeisawatertech.com	/var/www/nobleus-portal/web	portal-le-ssl.conf
billing.conf	billing.hargeisawatertech.com	/var/www/nobleus-portal/web	billing-le-ssl.conf
api.conf	api.hargeisawatertech.com	/var/www/nobleus-portal/web	api-le-ssl.conf
000-catch-all.conf	(bare IP / unmatched)	/var/www/html/blocked	—

Sample Portal Virtual Host (HTTP → HTTPS redirect)

```
/etc/apache2/sites-available/portal.conf
```



```

<VirtualHost *:80>
    ServerName portal.hargeisawatertech.com
    ServerAlias www.portal.hargeisawatertech.com
    # Redirect all HTTP to HTTPS (Certbot manages this block)
    RewriteEngine On
    RewriteRule ^ https://%{HTTP_HOST}%{REQUEST_URI} [L,R=301]
</VirtualHost>

<VirtualHost *:443>
    ServerName portal.hargeisawatertech.com
    DocumentRoot /var/www/nobleus-portal/web

    <Directory /var/www/nobleus-portal/web>
        Options -Indexes +FollowSymLinks
        AllowOverride All
        Require all granted
    </Directory>

    # SSL — managed by Certbot
    SSLEngine on
    SSLCertificateFile /etc/letsencrypt/live/portal.hargeisawatertech.com/fullchain.pem
    SSLCertificateKeyFile /etc/letsencrypt/live/portal.hargeisawatertech.com/privkey.pem
    Include /etc/letsencrypt/options-ssl-apache.conf

    # Security headers
    Header always set Strict-Transport-Security "max-age=31536000; includeSubDomains"
    Header always set X-Content-Type-Options "nosniff"
    Header always set X-Frame-Options "SAMEORIGIN"

    ErrorLog ${APACHE_LOG_DIR}/portal-error.log
    CustomLog ${APACHE_LOG_DIR}/portal-access.log combined
</VirtualHost>

```

Enabling Sites & Reloading Apache

```
bash — enable vhosts
```

```
ubuntu@hwa-portal:~$ sudo a2ensite portal.conf billing.conf api.conf 000-catch-all.conf
```

```
Enabling site portal.
```

```
Enabling site billing.
```

```
Enabling site api.
```

```
Enabling site 000-catch-all.
```

```
ubuntu@hwa-portal:~$ sudo apache2ctl configtest
```

```
Syntax OK
```

```
ubuntu@hwa-portal:~$ sudo systemctl reload apache2
```

SSL Certificate Management (Certbot)

```
bash — certbot operations
```

```
ubuntu@hwa-portal:~$ # Initial certificate issuance for all domains
ubuntu@hwa-portal:~$ sudo certbot --apache -d portal.hargeisawatertech.com \
-d billing.hargeisawatertech.com \
-d api.hargeisawatertech.com

ubuntu@hwa-portal:~$ # Test auto-renewal (dry run)
ubuntu@hwa-portal:~$ sudo certbot renew --dry-run
Simulating renewal of an existing certificate for portal.hargeisawatertech.com
...
Congratulations, all simulated renewals succeeded:
/etc/letsencrypt/live/portal.hargeisawatertech.com/fullchain.pem (success)

ubuntu@hwa-portal:~$ # Check cron entry (Certbot installs this automatically)
ubuntu@hwa-portal:~$ sudo cat /etc/cron.d/certbot
0 */12 * * * root test -x /usr/bin/certbot -a \! -d /run/systemd/system && perl -e 'sleep int(rand(43200))' && certbot -q renew
```

CATCH-ALL VIRTUAL HOST — ALWAYS KEEP ENABLED

The `000-catch-all.conf` virtual host intercepts any HTTP request that does not match a named `ServerName`. This prevents direct access to the application via the raw EC2 IP address (e.g. `http://54.x.x.x/`), which could bypass security headers and expose internal details. It returns a generic `403 Forbidden` for all bare-IP or unrecognised hostname requests.



Database — Local MySQL & Remote PostgreSQL

Schema overview, migration management, and remote connectivity

6.1 Local MySQL Database

The portal's primary data store is a MySQL 8.0 database named `nobleus_db` running locally on the EC2 instance at `127.0.0.1:3306`. The MySQL user `nobleus_app` is restricted to localhost connections only. Schema is managed via Yii 2 migrations.

<code>meter_reading_raw</code>	All raw telemetry records received from ThingsBoard webhooks	<code>id</code> , <code>meter_id</code> , <code>ts</code> (epoch ms)
<code>billing_ledger</code>	Processed billing entries derived from raw readings; one record per billing period per meter	<code>id</code> , <code>meter_id</code> , <code>billing_month</code>
<code>billing_audit_logs</code>	Full audit trail of every attempt to push a billing entry to the Core Billing API	<code>id</code> , <code>ledger_id</code> , <code>attempted_at</code>
<code>meters</code>	Registered meter inventory — maps ThingsBoard device IDs to HWA account numbers	<code>id</code> , <code>tb_device_id</code> , <code>account_n</code>
<code>meter_assignments</code>	Tracks which customer account a meter is assigned to over time (history-aware)	<code>id</code> , <code>meter_id</code> , <code>customer_id</code> ,

Running Migrations

```
ubuntu@hwa-portal:~$ cd /var/www/nobleus-portal
ubuntu@hwa-portal:/var/www/nobleus-portal$ php yii migrate
Yii Migration Tool (based on Yii v2.0.x)
```

```
Total 3 new migrations to be applied:
  m250601_000001_create_meter_reading_raw
  m250601_000002_create_billing_ledger
  m250601_000003_create_billing_audit_logs
Apply the above migrations? (yes|no) [no]: yes
*** applying m250601_000001_create_meter_reading_raw
  > create table {%meter_reading_raw%} ... done (time: 0.012s)
*** applied m250601_000001_create_meter_reading_raw (time: 0.013s)
*** applying m250601_000002_create_billing_ledger ... done
*** applying m250601_000003_create_billing_audit_logs ... done

Migrated up successfully.
```

Checking the Failed Push Queue

SQL — failed billing push records

-- View all billing ledger entries that failed to push to Core API

```
SELECT
  bl.id,
  bl.meter_id,
  bl.billing_month,
  bl.billing_date,
  bl.consumption_m3,
  bl.push_status,
  bal.attempted_at,
  bal.http_status,
  bal.response_body
FROM billing_ledger bl
LEFT JOIN billing_audit_logs bal ON bal.ledger_id = bl.id
WHERE bl.push_status = 'failed'
ORDER BY bal.attempted_at DESC
LIMIT 50;
```

6.2 Remote PostgreSQL (HWA On-Premise)

The Core Billing API writes billing records into an on-premise PostgreSQL database. The portal does not connect to PostgreSQL directly; the Core Billing REST API acts as the intermediary. However, sysadmins must ensure TCP 5432 outbound is reachable for troubleshooting and that the FortiGate VIP is operational.

Public VIP (FortiGate)	102.218.58.66
Internal Host (LAN)	10.10.50.120
Port	5432
Database	HWA_DATA
User	openpg
SSL Mode	require (self-signed cert, verify-full not enforced)

Verifying Remote PostgreSQL Reachability

bash — connectivity check from EC2

```
ubuntu@hwa-portal:~$ # Step 1: Verify TCP port is reachable
ubuntu@hwa-portal:~$ nc -vz 102.218.58.66 5432
Connection to 102.218.58.66 5432 port [tcp/postgresql] succeeded!

ubuntu@hwa-portal:~$ # Step 2: Attempt psql connection
ubuntu@hwa-portal:~$ psql -h 102.218.58.66 -p 5432 -U openpg -d HWA_DATA -c "SELECT version();"
Password for user openpg:
      version
-----
PostgreSQL 14.x on x86_64-pc-linux-gnu, compiled by ...
(1 row)
```



AWS SECURITY GROUP OUTBOUND RULE REQUIRED

Ensure the EC2 instance's security group has an **OUTBOUND RULE ALLOWING TCP 5432 TO 102.218.58.66/32**. By default, AWS security groups deny all outbound traffic if a custom outbound rule is specified. Missing this rule causes silent connection timeouts when the portal attempts to push billing data.



Billing Date Policy — The 28th Rule

How the system assigns official billing dates to meter readings



COMPANY POLICY — BILLING DATE STANDARDISATION

By HWA billing policy, **ALL METER READINGS USE THE 28TH OF THE READING'S BILLING MONTH AS THE OFFICIAL BILLING DATE**, regardless of when within the month the reading was actually recorded or processed. This ensures consistent billing cycles and simplifies reconciliation with the HWA Core Billing system.

Implementation in Code

The 28th-of-month rule is implemented in two places in the application:

Bulk Action (`actionPostToBilling`): When posting a batch of readings for a given month, the billing date is computed as:

PHP — bulk billing date assignment

```
// $month is in 'YYYY-MM' format (e.g. '2026-06')
// Always append '-28' to get the standardised billing date
$billingDate = $month . '-28';

// Example: $month = '2026-06' → $billingDate = '2026-06-28'
$payload = [
    'accountNo' => $reading->account_no,
    'readingDate' => $billingDate,
    'reading' => $reading->value_m3,
];
```

Single Import (`actionImport`): When importing individual readings, the billing date is derived from the reading's own timestamp, replacing the day component with 28:

PHP — single reading billing date

```
// Derive billing date: take reading's year-month, fix day to 28
$billingDate = date('Y-m', $reading->ts) . '-28';

// If reading was on 2026-07-15 09:30:00 UTC,
// billingDate becomes '2026-07-28'
```

Billing Date Behaviour Matrix

Reading on the 28th	2026-06-28	2026-06-28	No special note appended
Reading on another day	2026-06-15	2026-06-28	Description appended: <i>"Reading date adjusted to billing"</i>
Reading on last day of month	2026-06-30	2026-06-28	Description appended noting original reading date
No reading for month	—	—	Entry skipped entirely; no billing record pushed
Zero reading received	Any date	Month-28	Zero-consumption record pushed; flagged in audit log

i

FEBRUARY HANDLING

February always has at least 28 days (1-28), so the 28th always exists and is valid for any year. No special leap-year logic is required for billing date generation.



Deployment & Upgrade Procedures

First-time server setup and rolling code upgrades

8.1 First-Time Setup

Follow these steps in sequence on a freshly provisioned Ubuntu 22.04 EC2 instance. Ensure all prerequisites from Section 3 are installed before beginning.

Clone the repository

Clone the application code to the web root using your deploy key or HTTPS token.

```
bash
```

```
ubuntu@hwa-portal:~$ sudo mkdir -p /var/www/nobleus-portal
ubuntu@hwa-portal:~$ sudo chown ubuntu:ubuntu /var/www/nobleus-portal
ubuntu@hwa-portal:~$ git clone git@github.com:hwa-tech/nobleus-portal.git /var/www/nobleus-portal
Cloning into '/var/www/nobleus-portal'...
remote: Enumerating objects: 1842, done.
remote: Counting objects: 100% (1842/1842), done.
Receiving objects: 100% (1842/1842), done.
```

Create and populate the .env file

Copy the example file and fill in all production credentials as described in Section 4.

```
bash
```

```
ubuntu@hwa-portal:~$ cp /var/www/nobleus-portal/.env.example /var/www/nobleus-portal/.env
ubuntu@hwa-portal:~$ nano /var/www/nobleus-portal/.env
# Edit all values, then save
ubuntu@hwa-portal:~$ sudo chown www-data:www-data /var/www/nobleus-portal/.env
ubuntu@hwa-portal:~$ sudo chmod 640 /var/www/nobleus-portal/.env
```

Install PHP dependencies with Composer

```
bash
```

```
ubuntu@hwa-portal:~$ cd /var/www/nobleus-portal && composer install --no-dev --optimize-autoloader
Installing dependencies from lock file (including require-dev)
Package operations: 94 installs, 0 updates, 0 removals
 - Installing yiisoft/yii2 (2.0.49): Extracting archive
...
Generating optimized autoload files
Generated optimized autoload files containing 2534 classes
```

Run database migrations

```
bash
```

```
ubuntu@hwa-portal:/var/www/nobleus-portal$ php yii migrate --interactive=0
Migrated up successfully.
```

Set correct file permissions

```
bash
```

```
ubuntu@hwa-portal:~$ sudo chown -R www-data:www-data /var/www/nobleus-portal
ubuntu@hwa-portal:~$ sudo chmod -R 755 /var/www/nobleus-portal
ubuntu@hwa-portal:~$ sudo chmod -R 775 /var/www/nobleus-portal/runtime
ubuntu@hwa-portal:~$ sudo chmod -R 775 /var/www/nobleus-portal/web/assets
```

Enable Apache virtual hosts and reload

```
bash
```

```
ubuntu@hwa-portal:~$ sudo a2enmod rewrite ssl headers deflate
ubuntu@hwa-portal:~$ sudo a2ensite portal.conf billing.conf api.conf 000-catch-all.conf
ubuntu@hwa-portal:~$ sudo apache2ctl configtest && sudo systemctl reload apache2
Syntax OK
```

8.2 Code Upgrade (Rolling Update)

Use the following procedure for routine code updates. The application remains online during the upgrade; Apache is only reloaded (not restarted) at the end.

Pull latest code from Git

```
bash
```

```
ubuntu@hwa-portal:~$ cd /var/www/nobleus-portal && sudo -u www-data git pull origin main
Updating a1b2c3d..f4e5d6c
Fast-forward
 controllers/BillingController.php | 14 ++++---
 models/MeterReading.php           | 8 ++--
 2 files changed, 22 insertions(+), 0 deletions(-)
```

Update Composer dependencies (if composer.json changed)

```
bash
```

```
ubuntu@hwa-portal:/var/www/nobleus-portal$ sudo -u www-data composer install --no-dev --optimize-autoloader
```

Apply any new database migrations

```
bash

ubuntu@hwa-portal:/var/www/nobleus-portal$ sudo -u www-data php yii migrate --interactive=0
No new migrations found. Your system is up-to-date.
```



Reload Apache to pick up any new PHP OpCache changes

```
bash

ubuntu@hwa-portal:~$ sudo systemctl reload apache2
# No output = success
```



Logs & Monitoring

Log file locations, tailing commands, and database audit queries

The portal generates logs at three levels: the Apache web server layer (access and error), the Yii 2 application layer (runtime log), and the database audit layer (billing push history). Monitoring all three is essential for diagnosing integration issues quickly.

Apache Access Log (Portal)	/var/log/apache2/portal-access.log	All HTTP requests — method, URI, status code, response size, user-agent
Apache Error Log (Portal)	/var/log/apache2/portal-error.log	PHP fatal errors, Apache module errors, SSL errors
Yii App Log	/var/www/nobleus-portal/runtime/logs/app.log	Application-level info, warnings, errors; billing push results; ThingsBoard
Catch-All Access Log	/var/log/apache2/catchall-access.log	All requests arriving via bare IP or unrecognised hostname — security

Tailing Logs in Real-Time

```
bash — live log monitoring
```



```
ubuntu@hwa-portal:~$ # Follow application log (Yii 2)
ubuntu@hwa-portal:~$ tail -f /var/www/nobleus-portal/runtime/logs/app.log
2026-07-08 09:15:32 [info][application] ThingsBoard webhook received: device tb-device-00142, value 1247.8 m³
2026-07-08 09:15:33 [info][application] Billing push SUCCESS: account 4521-A, billing_date 2026-06-28, http_status 200
2026-07-08 09:15:34 [warning][application] Billing push FAILED: account 4522-B, http_status 422, response: {"error":"duplicate_entry"}

ubuntu@hwa-portal:~$ # Follow Apache error log (PHP + Apache errors)
ubuntu@hwa-portal:~$ tail -f /var/log/apache2/portal-error.log

ubuntu@hwa-portal:~$ # Combined multi-log tail
ubuntu@hwa-portal:~$ tail -f /var/www/nobleus-portal/runtime/logs/app.log \
/var/log/apache2/portal-error.log
```

Billing Audit Log Query — Failed Pushes

SQL — billing audit investigation

```
-- Recent billing push failures with full response details
SELECT
  bal.id          AS audit_id,
  bl.meter_id,
  bl.billing_month,
  bl.billing_date,
  bal.attempted_at,
  bal.http_status,
  LEFT(bal.response_body, 200) AS response_snippet,
  bal.success
FROM    billing_audit_logs bal
INNER JOIN billing_ledger bl ON bl.id = bal.ledger_id
WHERE   bal.success = 0
AND     bal.attempted_at >= NOW() - INTERVAL 7 DAY
ORDER BY bal.attempted_at DESC;
```



PRO TIP — DISK SPACE MONITORING

Set up a cron-based disk usage alert. The Yii application log and Apache access logs can fill the EBS volume if billing operations are very frequent. A simple alert: `df -h / | awk 'NR==2{if($5+0>80) print "DISK USAGE ABOVE 80%: "$5}'` run nightly from root's crontab provides early warning.



Troubleshooting Reference

Common issues, their causes, and step-by-step remediation

The table below covers the most frequently encountered operational issues. For each issue, the most likely cause is listed alongside the diagnostic command and recommended fix. Always check the Yii app log first (`runtime/logs/app.log`) before escalating.

Billing push status shows "failed" in <code>billing_ledger</code>	Expired <code>CORE_API_TOKEN</code> (JWT), network timeout to FortiGate, or
Apache SSL certificate error — browsers show "cert expired" warning	Certbot auto-renewal cron failed (e.g., port 80 blocked during
504 Gateway Timeout on bulk billing post action	Large number of records being pushed in a single HTTP requ
Meter readings not appearing in portal after ThingsBoard sends webhook	Invalid or missing <code>TB_WEBHOOK_TOKEN</code> causing 401 rejection; v
Readings appear as zero for multiple meters across a period	Meter device offline or not transmitting; ThingsBoard receivin
<code>billing_audit_logs</code> table is empty after bulk post action	No billing entries exist in <code>billing_ledger</code> for the selected month;



ESCALATION PATH

If an issue cannot be resolved with the steps above, escalate in order: (1) HWA IT Help Desk for Core Billing API issues, (2) AWS support for EC2/networking issues, (3) ThingsBoard vendor support for IoT platform issues, (4) Nobleus development team for application code issues. Always capture the relevant log extract before escalating.



Security Checklist

Mandatory and recommended security controls for production operation

The following checklist must be reviewed at initial deployment and re-verified at least every six months. Items marked with a red badge represent critical controls — any failure immediately increases attack surface or data exposure risk.

Token & Credential Management

- 
- Rotate `CORE_API_TOKEN` (JWT) every 6 months.** Coordinate with HWA IT to generate a new token from the Core Billing API. Update `.env` and reload Apache. Log the rotation date in the ops log. **CRITICAL**
- 
- Rotate `TB_WEBHOOK_TOKEN` every 6 months.** Generate a new strong random token (32+ chars), update in both ThingsBoard rule engine and the portal `.env`, then reload Apache. **CRITICAL**



Never commit `.env` to any Git repository. Verify `.env` is listed in `.gitignore`. Periodically run `git log --all --full-history -- .env` to confirm it has never been committed. **CRITICAL**



Set `.env` file permissions to `640`, owned by `www-data:www-data`. Command: `sudo chmod 640 /var/www/nobleus-portal/.env && sudo chown www-data:www-data /var/www/nobleus-portal/.env`

Application Configuration



Ensure `YII_DEBUG=false` in production. Debug mode exposes full stack traces, configuration details, and SQL queries in the browser — a severe information disclosure risk. **CRITICAL**



Ensure `YII_ENV=prod` in production. This disables development-only behaviours, profiling panels, and Gii code generation scaffolding.



Ensure `CORE_API_INSECURE_SSL=0` (or unset) in production. Setting this to `1` disables TLS verification on all Core Billing API calls, exposing all JWT tokens to MITM attacks. **CRITICAL**

SSL & Transport Security



SSL auto-renewal cron is active and tested. Run `sudo certbot renew --dry-run` monthly to verify ACME renewal works. Verify `/etc/cron.d/certbot` exists.



HSTS header is set on all SSL virtual hosts. Confirm `Strict-Transport-Security` header appears in responses: `curl -I https://portal.hargeisawatertech.com | grep Strict`



HTTP-to-HTTPS redirect is enforced on all virtual hosts. Test: `curl -I http://portal.hargeisawatertech.com` must return `301 Moved Permanently` pointing to HTTPS.

Database Security



MySQL user `nobleus_app` is restricted to localhost connections only. Verify: `SELECT host, user FROM mysql.user WHERE user='nobleus_app';` — result must show `127.0.0.1` or `localhost` only.



MySQL is bound to `127.0.0.1` only (not `0.0.0.0`). Check `/etc/mysql/mysql.conf.d/mysqld.cnf` — `bind-address = 127.0.0.1` must be set. Port 3306 must not appear in EC2 security group inbound rules.

AWS & Network Security



Catch-all virtual host (`000-catch-all.conf`) is always enabled. This prevents bare-IP access. Test: `curl -I http://<EC2-`

IP>/ must return 403 , not a portal page.



AWS EC2 security group inbound rules: only ports 80 (HTTP) and 443 (HTTPS) open to 0.0.0.0/0 . SSH (port 22) must be restricted to the administrator's office IP range, never 0.0.0.0/0 . **CRITICAL**



AWS EC2 security group outbound rules: allow TCP 443 to 0.0.0.0/0 and TCP 5432 to 102.218.58.66/32 . Restrict all other outbound traffic where possible.



AWS EC2 instance is in a VPC with a defined network ACL. Ensure the NACL does not inadvertently expose internal services to the internet.

Operational Hygiene



Ubuntu unattended security updates are enabled. Verify: `sudo dpkg -l unattended-upgrades` — package must be installed and configured. Check `/etc/apt/apt.conf.d/50unattended-upgrades` .



EBS volume snapshots are scheduled (at minimum weekly). Configure AWS Backup or DLM (Data Lifecycle Manager) policy for the root EBS volume. Retain at least 4 snapshots.



MySQL backups are performed daily. Example cron: `0 2 * * * mysqldump -u root nobleus_db | gzip > /var/backups/nobleus-$(date +%F).sql.gz` . Verify backups can be restored.